

¿Qué matemáticas necesitas para el Machine Learning?

Los cuatro pilares de matemáticas aplicadas

César Galindo

Facultad de Empresariales, CDUP
Universidad Panamericana, Campus Mixcoac

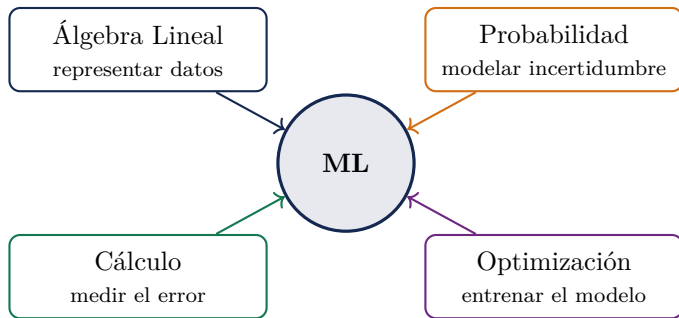
17 de septiembre de 2026

¿Qué es *machine learning*, en una frase?

Un algoritmo que **ajusta parámetros numéricos** hasta que sus predicciones se parecen a los datos que ya conocemos.

- Un filtro de spam, un traductor automático, un sistema de recomendación: todos siguen esta misma receta.
- No hay magia: hay **álgebra lineal, probabilidad, cálculo y optimización** trabajando juntos.

Los cuatro pilares



Cada pilar responde una pregunta distinta; juntos arman el algoritmo de entrenamiento.

I. Álgebra Lineal

el lenguaje de los datos

Todo dato es un vector

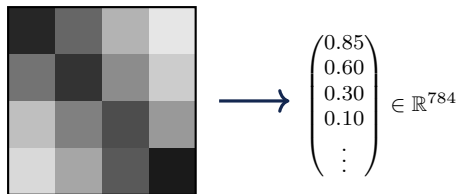


imagen (matriz de píxeles)

- Un documento de texto $\rightarrow$ un vector de frecuencias de palabras.
- Un usuario de una app $\rightarrow$ un vector de características (edad, clics, tiempo en la app, ...).

Lo que ya conocen de sus cursos

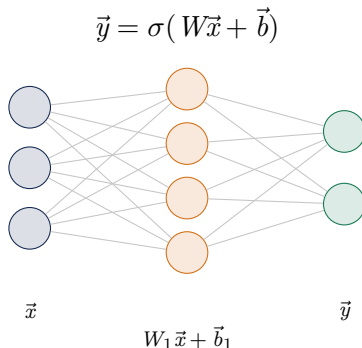
Vectores, matrices, combinaciones lineales: eso ya lo vieron o lo están viendo en Álgebra Lineal.

Medir parecido: producto punto y normas

$$\vec{u} \cdot \vec{v} = \sum_i u_i v_i, \quad \|\vec{u}\| = \sqrt{\vec{u} \cdot \vec{u}}$$

- El producto punto normalizado da la **similitud coseno**: la base de casi todo buscador o recomendador.
- “¿Qué tan parecidos son estos dos usuarios / estas dos canciones / estos dos textos?” se responde con geometría.

Una red neuronal es multiplicación de matrices



- $\vec{x}$: los datos de entrada (vector). W : los parámetros a aprender (matriz). σ : una función no lineal simple, número por número.

Apilar varias de estas transformaciones es, literalmente, una red neuronal profunda.

Eigenvalores, SVD y reducción de dimensión

- Muchos datos reales viven, en el fondo, en pocas direcciones importantes.
- **PCA** (análisis de componentes principales) usa eigenvectores para encontrar esas direcciones.
- Resultado: pasar de miles de variables a unas cuantas, sin perder casi información.

II. Probabilidad y Estadística

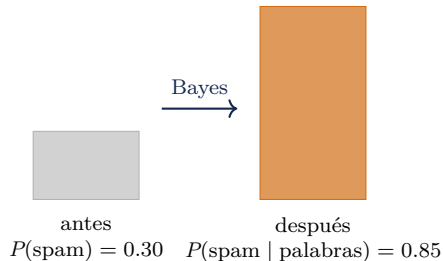
modelar la incertidumbre

Los datos nunca son perfectos

- Ningún modelo predice con certeza absoluta: predice **distribuciones de probabilidad**.
- “80% de probabilidad de que este correo sea spam” es más honesto que “sí o no”.
- Por eso el ML se apoya en variables aleatorias, esperanza y varianza desde el primer día.

Ejemplo: un filtro de spam con Bayes

$$P(\text{spam} \mid \text{palabras}) = \frac{P(\text{palabras} \mid \text{spam}) P(\text{spam})}{P(\text{palabras})}$$



- El teorema de Bayes convierte “qué tan comunes son estas palabras en el spam” en “qué tan probable es que este correo sea spam”.
- Es uno de los clasificadores más simples que existen y sigue funcionando bien hoy.

El problema central: sesgo y varianza



- Un modelo **demasiado simple**: no captura el patrón real (*sesgo alto*).
- Un modelo **demasiado complejo**: memoriza el ruido en vez del patrón (*sobreajuste*, varianza alta).
- Encontrar el punto intermedio es, en esencia, un problema estadístico.

III. Cálculo

cómo aprende la máquina

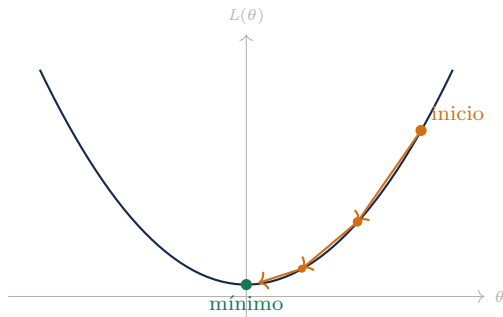
Aprender = minimizar un error

- Se define una función de error (o *pérdida*) $L(\theta)$ que mide qué tan mal predice el modelo.
- “Entrenar” un modelo es, literalmente, resolver

$$\min_{\theta} L(\theta).$$

- Esto ya lo conocen: es optimización de una función, como en Cálculo.

El gradiente indica hacia dónde bajar



$\theta \leftarrow \theta - \eta \nabla L(\theta)$: en cada paso, muévete en la dirección de mayor descenso.

La regla de la cadena entrena redes enteras

- Una red neuronal es una composición de funciones: $L(f_3(f_2(f_1(\vec{x}))))$.
- La **regla de la cadena** de Cálculo II/III permite calcular el gradiente de esa composición capa por capa.
- A ese algoritmo se le llama *backpropagation*: no es más que la regla de la cadena, aplicada de forma sistemática.

IV. Optimización

el motor del entrenamiento

Descenso de gradiente (estocástico)

- Calcular el gradiente con **todos** los datos en cada paso es carísimo.
- Solución: usar solo un pedacito de los datos (un *mini-lote*) en cada paso → **SGD**.
- Es ruidoso, pero en la práctica converge más rápido y generaliza mejor.

Convexo vs. no convexo



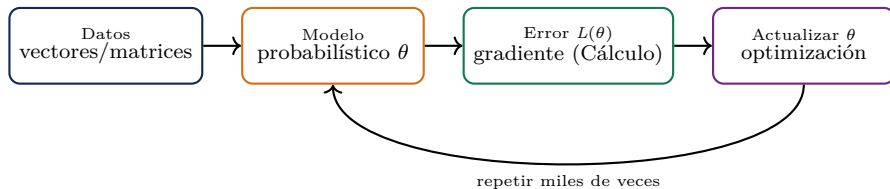
convexa: un único mínimo



no convexa: varios mínimos locales

- Si $L(\theta)$ es **convexa**: hay un único mínimo y el descenso de gradiente lo encuentra siempre.
- Las redes neuronales **no** son convexas: hay muchos mínimos locales.
- Sorpresa (activa) de investigación: en la práctica, casi todos esos mínimos funcionan razonablemente bien.

Cómo se conectan los cuatro pilares



Álgebra lineal (representar) $\rightarrow$ Probabilidad (modelar) $\rightarrow$ Cálculo (medir el error) $\rightarrow$ Optimización (corregir). Ese ciclo, repetido, es el entrenamiento.

¿Qué cursar y cuándo?

Semestre	Materia	Conexión con ML
1 ^{er} –2 ^o	Cálculo I–II	gradientes, funciones de error
2 ^o –3 ^{er}	Álgebra Lineal	representación de datos
3 ^{er} –4 ^o	Probabilidad	modelos, incertidumbre
4 ^o en adelante	Cálculo III / Optimización	entrenamiento, redes neuronales
4 ^o en adelante	Estadística inferencial	validación, sobreajuste

Complementos útiles, no imprescindibles al inicio: matemáticas discretas y teoría de grafos (redes, árboles de decisión); métodos numéricos (estabilidad, aproximación); y, por supuesto, programación.

Si quieren empezar ya

- Practiquen álgebra lineal con **operaciones**, no solo teoría: multiplicar matrices a mano ayuda a entender qué hace una red.
- En cálculo, presten atención a **gradiente** y **regla de la cadena**: son las dos ideas que más se reciclan.
- En probabilidad, no se salten **Bayes** ni **esperanza/varianza**: aparecen en todos lados.
- Programar (Python) no sustituye las matemáticas: las hace *tangibles*.

Algunas referencias clásicas

G. Strang, *Linear Algebra and Its Applications* | C. Bishop, *Pattern Recognition and Machine Learning* | James, Witten, Hastie, Tibshirani, *An Introduction to Statistical Learning*

Para llevarse

El machine learning no es una materia nueva:
es **álgebra lineal, probabilidad, cálculo y optimización**,
trabajando juntas.

Gracias

¿Preguntas?